

GIT y GITHUB

Del control de versiones al ecosistema de desarrollo colaborativo



BearsLoveBeer

Índice

1. Qué es Git.....	1
1.1. Breve historia: Un nacimiento por necesidad.....	1
1.2. La actualidad de Git.....	2
2. Instalación y configuración.....	4
2.1. Instalación y verificación.....	4
2.1.1. Configuración de identidad (obligatorio).....	4
2.1.2. Inicialización del proyecto local.....	4
2.1.3. Configuración de la rama principal (opcional pero recomendado).....	5
2.1.3.1. Porqué se cambió de master a main.....	5
2.1.4. El ciclo de trabajo básico (local).....	6
2.2. Los tres estados de git.....	6
2.2.1. Modificado (Modified).....	6
2.2.2. Preparado (Staged).....	6
2.2.3. Confirmado (Committed).....	7
2.2.4. Resumen del ciclo de vida.....	7
2.3. Flujo de trabajo básico.....	8
2.4. Git ignore (.gitignore).....	8
2.5. Historial y "viajes en el tiempo".....	9
2.6. Ramas (Branches).....	9
2.6.1. El concepto de las ramas.....	10
2.6.2. ¿Por qué son tan importantes?.....	10
2.7. Fusión (Merge) y conflictos.....	11
2.8. Herramientas avanzadas y seguridad.....	11
2.9. Resumen de comandos clave (Cheat Sheet).....	11
3. GITHUB.....	13
3.1. Los orígenes: El problema de compartir código.....	13
3.2. La revolución de GitHub.....	13
3.3. La era de Microsoft.....	14
3.4. El presente y futuro.....	14
3.5. Conceptos fundamentales: Git vs. GitHub.....	14
3.6. Conectar con el repositorio remoto (GitHub).....	14
3.7. Repositorios Remotos.....	15
3.8. Ignorar Archivos Innecesarios (.gitignore).....	15
3.9. Autenticación con GitHub (Claves SSH).....	15
3.9.1. Las Claves SSH.....	15
3.9.2. ¿Cómo funciona en la práctica?.....	16
3.9.3. Cómo procedemos para crear y utilizar SSH.....	17
3.10. Sincronización de cambios (trabajo en equipo).....	17
3.11. Clonación (Git Clone).....	17
3.12. Flujo de trabajo "Open source" (Fork y Pull request).....	18
3.13. Mantener un Fork actualizado.....	18

3.14. Documentación (Markdown).....	18
4. Otras herramientas de Github.....	19
4.1. Personalización del perfil (Profile README).....	19
4.1.1. Posibilidades y elementos clave.....	19
4.1.2. Dominio de Markdown (.md).....	19
4.1.2.1. Breve historia.....	20
4.1.2.2. ¿Por qué es el estándar en el desarrollo de software?.....	20
4.1.2.3. Sintaxis básica.....	21
4.1.2.4. Markdown en GitHub (GFM).....	22
4.1.2.4.1. Los "superpoderes" que añade GFM.....	22
4.2. GitHub Pages.....	22
4.2.1. Casos de uso ideales.....	22
4.2.2. La magia detrás: De Markdown a web.....	23
4.3. GitHub Actions (Automatización).....	23
4.3.1. ¿Cómo funciona la automatización? (Conceptos clave).....	23
4.3.2. Casos de uso más potentes.....	24
5. Bibliografía.....	25

1. Qué es Git.

Git es un **sistema de control de versiones distribuido**. En términos prácticos, funciona como una "máquina del tiempo" o un sistema de autoguardado de un videojuego extremadamente avanzado para tu código y archivos.

Su función principal es registrar cada cambio o ciertos cambios que se hacen en un proyecto a lo largo del tiempo. Esto permite a los desarrolladores volver a versiones anteriores si algo deja de funcionar, experimentar con nuevas ideas de forma segura (mediante las ramas o *branches* las cuales hablaremos más adelante) y colaborar con decenas o miles de personas al mismo tiempo sin que nadie sobrescriba el trabajo del otro.

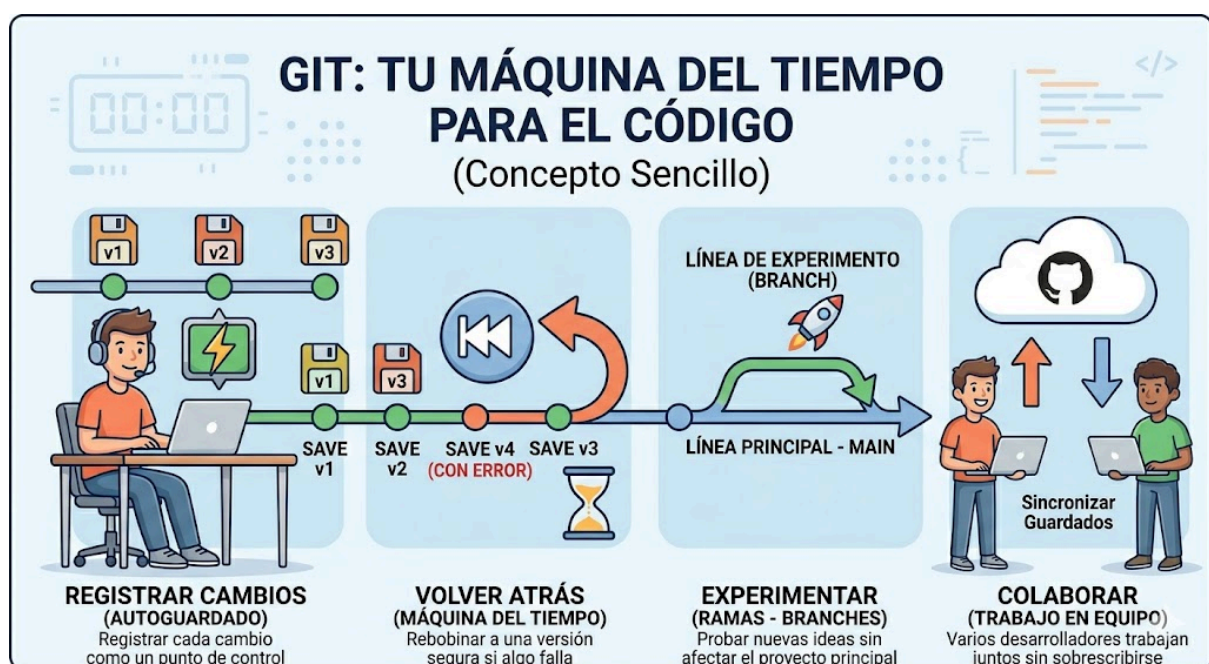


Figura 1. Git: Tu máquina del tiempo del código.

1.1. Breve historia: Un nacimiento por necesidad.

Git nació en abril de **2005** en medio de una crisis, de la mano de **Linus Torvalds** (el creador del sistema operativo Linux).

Durante años, la inmensa comunidad de desarrolladores de Linux utilizaba un software de control de versiones privativo llamado BitKeeper. Sin embargo, tras una disputa sobre la ingeniería inversa y las licencias, la empresa creadora de BitKeeper revocó el acceso gratuito a la comunidad de Linux.



Figura 2. Fotografía de Linus Torvalds: la mente detrás de Linux y Git.

Torvalds evaluó las alternativas de código abierto que existían en ese momento (como CVS o SVN) y concluyó que ninguna era lo suficientemente rápida o capaz de manejar un proyecto tan masivo y descentralizado como Linux. Así que decidió crear el suyo propio.

Lo diseñó desde cero enfocado en tres pilares:

- **Velocidad extrema.**
- **Diseño distribuido** (donde cada programador tiene una copia local exacta de todo el historial del proyecto, no solo un servidor central).
- **Soporte robusto para el desarrollo no lineal** (miles de ramas paralelas fusionándose constantemente).

Torvalds programó la primera versión funcional de Git en apenas un par de semanas.

1.2. La actualidad de Git.

Hoy en día, Git ha ganado la batalla del control de versiones por goleada. Ya no es simplemente una herramienta popular; es el **estándar absoluto y universal** en la industria del desarrollo de software.

- **Omnipresencia:** Desde desarrolladores independientes que hacen scripts por afición hasta corporaciones gigantes como Google, Microsoft o Netflix, prácticamente todo el mundo usa Git. Es un requisito fundamental para casi cualquier puesto de programación.
- **Plataformas masivas:** Git es el motor invisible que impulsa gigantes de la industria como **GitHub** (comprado por Microsoft), **GitLab** y **Bitbucket**. Estas plataformas tomaron la tecnología de Git y le añadieron una interfaz web, herramientas de colaboración y gestión de proyectos, convirtiéndose en el centro de operaciones global del código abierto y privado.

- **El corazón de la automatización (DevOps):** Git ha evolucionado más allá de ser solo un almacén de código. En la actualidad, es la pieza central de la automatización. A través de metodologías como GitOps o CI/CD (Integración y Despliegue Continuo), el simple acto de guardar un cambio con Git puede desencadenar servidores que prueban automáticamente el código, detectan errores de seguridad y lo publican directamente en internet.

2. Instalación y configuración.

Para comenzar desde cero en una nueva estación de trabajo con tu primer proyecto, es fundamental establecer una configuración base sólida y comprender el flujo de trabajo inicial.

2.1. Instalación y verificación.

Antes de nada, debes asegurarte de tener Git instalado en tu nueva máquina.

- **Instalación:** Si usas Linux o Mac, es probable que ya lo tengas instalado, aunque quizá no la última versión. En Windows, puedes descargar el instalador oficial.
- **Verificación:** Abre tu terminal (o Git Bash en Windows) y ejecuta el siguiente comando para verificar la instalación y la versión:

```
git --version
```

Si el comando devuelve un número de versión (ej. 2.40.0), estás listo para continuar.

2.1.1. Configuración de identidad (obligatorio).

Git necesita saber quién eres para asociar los cambios que hagas a tu identidad. Esto es **crítico** en una máquina nueva, ya que sin esto no podrás realizar "commits" (guardar cambios).

Ejecuta los siguientes comandos (configuración global para que aplique a todos tus proyectos futuros):

- **Configurar nombre de usuario:**

```
git config --global user.name "Tu Nombre"
```
- **Configurar correo electrónico:**

```
git config --global user.email "tu@email.com"
```

Estos datos quedarán inmutables en cada confirmación de cambios que realices. Puedes verificar tu configuración con

```
git config --list
```

.

2.1.2. Inicialización del proyecto local.

Ahora crearás tu proyecto en tu ordenador.

Crear directorio: Crea una carpeta para tu proyecto y entra en ella desde la terminal:

```
mkdir mi-primer-proyecto
```



```
cd mi-primer-proyecto
```

Iniciar Git: Ejecuta el comando para transformar esa carpeta en un repositorio: `git init`

- Esto crea una carpeta oculta `.git` donde se almacenará todo el historial de versiones.

2.1.3. Configuración de la rama principal (opcional pero recomendado).

En el sistema de control de versiones Git (la tecnología sobre la que se construye GitHub), "master" era el **nombre predeterminado** que se le asignaba automáticamente a la primera rama al crear un nuevo repositorio.

Históricamente, esta rama actuaba como la "fuente de la verdad" del proyecto. Era el lugar donde residía el código estable, la versión definitiva que se enviaba a producción o la base principal sobre la que se estructuraba el resto del desarrollo.

Hoy en día, el estándar es "main". Puedes configurar Git para que use "main" por defecto en los nuevos repositorios o cambiar el nombre manualmente al iniciar:

Comando:

`git config --global init.defaultBranch main`

2.1.3.1. Porqué se cambió de master a main.

El cambio, liderado por GitHub a finales de 2020 y posteriormente adoptado por la industria en general (incluyendo GitLab, Bitbucket y el propio proyecto de software Git), se debió a un esfuerzo consciente por utilizar un **lenguaje más inclusivo** en la tecnología.

- **El contexto histórico:** En la informática, los términos *master* y *slave* (maestro/esclavo) se han utilizado durante décadas para describir componentes de hardware, bases de datos y arquitecturas de red donde un proceso controla a otro. Estos términos arrastran connotaciones directas con la esclavitud.
- **El origen en Git:** Aunque en el caso específico de Git, el término master provenía originalmente de "master copy" (como la grabación original o cinta maestra en la industria de la música) y no tenía una relación directa con la arquitectura maestro/esclavo, la comunidad tecnológica decidió que la palabra en sí misma seguía generando asociaciones históricas dolorosas e innecesarias.
- **El consenso por main:** Se buscó una alternativa que fuera descriptiva y neutral. Se eligió main (principal) porque cumple perfectamente ese objetivo.

2.1.4. El ciclo de trabajo básico (local).

Git trabaja con tres estados: directorio de trabajo, área de preparación (stage) y directorio Git (commits).

- **Crear un archivo:** Crea un archivo de prueba, por ejemplo "README.md" o un script simple.
- **Verificar estado:** Usa `git status` para ver que tienes archivos "untracked" (sin seguimiento).

Añadir al Stage (Preparar): Para indicar que quieres incluir este archivo en la próxima "foto" o versión:

`git add README.md`

- (O usa `git add .` para añadir todo).

Hacer Commit (Confirmar): Guarda la versión con un mensaje descriptivo:

`git commit -m "Mi primer commit: añadir readme"`

- Esto crea una instantánea permanente de tu proyecto en ese momento.

2.2. Los tres estados de git.

Para entender cómo funciona Git, es fundamental conocer sus **tres estados principales**. Git no guarda los cambios automáticamente en el momento en que los haces; en su lugar, requiere que pases los archivos a través de tres fases distintas. Esto te da un control total sobre qué cambios exactos quieres incluir en cada versión de tu proyecto.

Aquí tienes los tres estados explicados de forma sencilla.

2.2.1. Modificado (Modified).

Significa que has hecho cambios en un archivo (has añadido código, borrado texto, etc.), pero **aún no lo has guardado** en la base de datos de Git.

- **En la práctica:** Es como estar trabajando en un documento de texto. Has escrito un párrafo nuevo, pero todavía no has pulsado el botón de "Guardar". Git sabe que el archivo ha cambiado desde la última vez, pero de momento esos cambios solo existen en tu área de trabajo actual.

2.2.2. Preparado (Staged).

Significa que has marcado un archivo modificado para que **se incluya en el próximo guardado** (commit).

- **En la práctica:** Imagina que estás metiendo cosas en una caja antes de enviarla por correo. En este estado, estás decidiendo exactamente qué archivos modificados van dentro de la caja. Puedes modificar 10 archivos, pero decidir que solo vas a "preparar" 3 de ellos porque son los únicos que están listos.

2.2.3. Confirmado (Committed).

Significa que los datos están **guardados de forma segura** en tu base de datos local de Git (tu repositorio).

- **En la práctica:** Es el equivalente a precintar la caja, ponerle una etiqueta que describe lo que hay dentro (el mensaje del commit) y guardarla en el almacén. A partir de este momento, se ha creado un punto de control en la historia de tu proyecto al que siempre podrás volver en el futuro.

2.2.4. Resumen del ciclo de vida.

Estos tres estados se relacionan directamente con las tres secciones principales de un proyecto de Git. Aquí tienes cómo fluye el trabajo:

Estado	Dónde se encuentra el archivo	Acción o comando para llegar ahí
Modificado	Directorio de Trabajo (<i>Working Directory</i>)	Editas tus archivos de código normalmente.
Preparado	Área de Preparación (<i>Staging Area</i> o <i>Index</i>)	Ejecutas el comando <code>git add <archivo></code>
Confirmado	Directorio de Git (<i>Local Repository</i>)	Ejecutas el comando <code>git commit -m "Mensaje"</code>

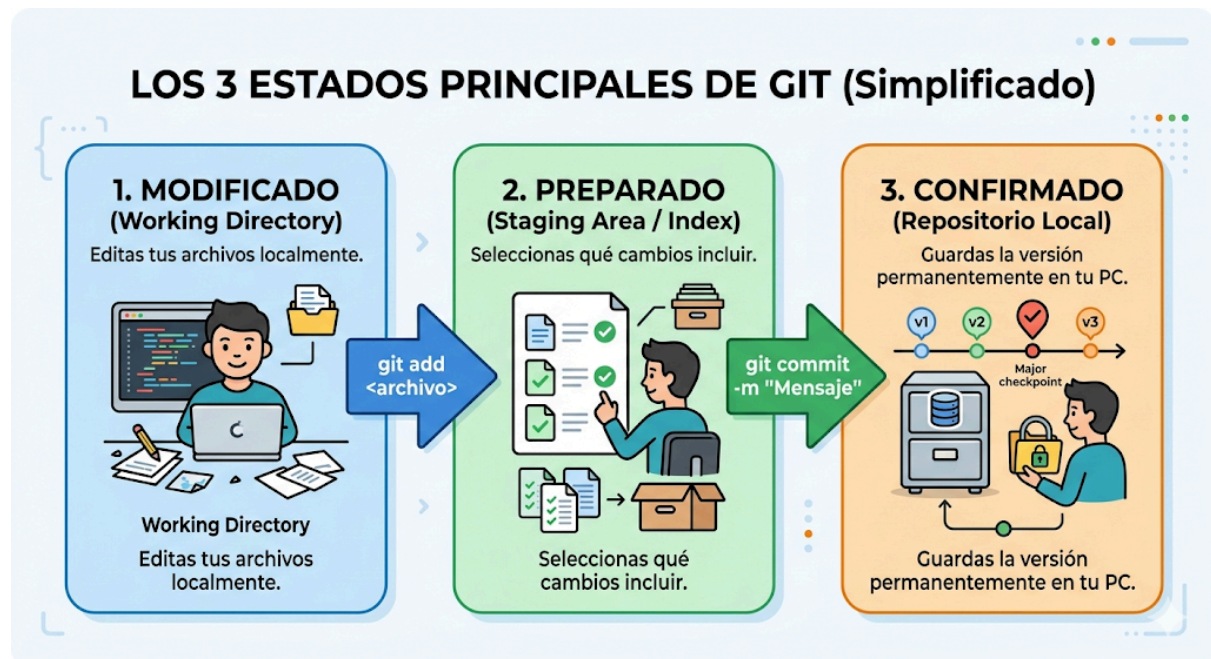


Figura 3. Flujo de trabajo básico de Git: Los tres estados.

El flujo de trabajo estándar de Git siempre sigue este mismo orden: primero modificas los archivos en tu directorio de trabajo, luego preparas solo los que quieres guardar, y finalmente confirmas esos cambios para que queden registrados en el historial de forma permanente.

2.3. Flujo de trabajo básico.

Este es el ciclo que repetirás constantemente al programar:

1. **Inicializar:** `git init` convierte una carpeta normal en un repositorio de Git, creando la carpeta oculta `.git`.
2. **Comprobar Estado:** `git status` es vital. Te dice qué archivos son nuevos ("untracked"), cuáles han sido modificados y cuáles están listos en el área de preparación.
3. **Preparar (Add):** `git add <archivo>` mueve los cambios del directorio de trabajo al área de preparación (stage). Si usas `git add .` añades todo lo que haya cambiado.
4. **Confirmar (Commit):** `git commit -m "Mensaje"` toma lo que está en el "stage" y lo guarda como una versión permanente en la historia. **Importante:** Cada commit debe tener un mensaje descriptivo. Cada commit genera un identificador único (Hash).

2.4. Git ignore (.gitignore).

No todo debe subirse al repositorio (ej. archivos temporales del sistema, claves privadas o carpetas de compilación).

- **Cómo funciona:** Creas un archivo llamado `.gitignore` en la raíz y escribes los nombres o extensiones a ignorar (ej. `*.log`, `.DS_Store`). Git dejará de rastrearlos automáticamente.

2.5. Historial y "viajes en el tiempo".

Git permite ver qué pasó y volver atrás.

- **Ver historial:** `git log` muestra la lista de commits (autor, fecha, mensaje). Se puede usar `git log --graph` para verlo como un gráfico de ramas.
- **Ver diferencias:** `git diff` muestra qué líneas exactas han cambiado en el código antes de hacer un commit.
- **Moverse en el tiempo (Checkout):**
 - `git checkout <Hash>`: Te lleva al estado exacto de un commit antiguo. **Cuidado:** Entrar en modo "Detached HEAD" (cabeza desconectada), lo que significa que no estás en ninguna rama y los cambios que hagas aquí pueden perderse si no creas una nueva rama.
 - `git checkout <archivo>`: Descarta los cambios locales de un archivo y lo restaura a como estaba en el último commit.

2.6. Ramas (Branches).

Las ramas permiten crear líneas paralelas de trabajo (ej. una rama para una nueva funcionalidad login y otra para arreglar un error fix-bug), sin afectar al código principal (main).

Imagina que estás escribiendo un libro. Tienes el manuscrito original, que está perfecto y listo para publicarse. De repente, se te ocurre una idea brillante para un final alternativo, pero tienes miedo de reescribir las últimas páginas y arruinar la versión original si la nueva idea no funciona.

¿Qué harías? Harías una **copia del documento**, probarías tu nuevo final en esa copia y, si te gusta cómo queda, lo juntamos con el libro original. Si no te gusta, simplemente tiras la copia a la basura y tu manuscrito original sigue intacto.

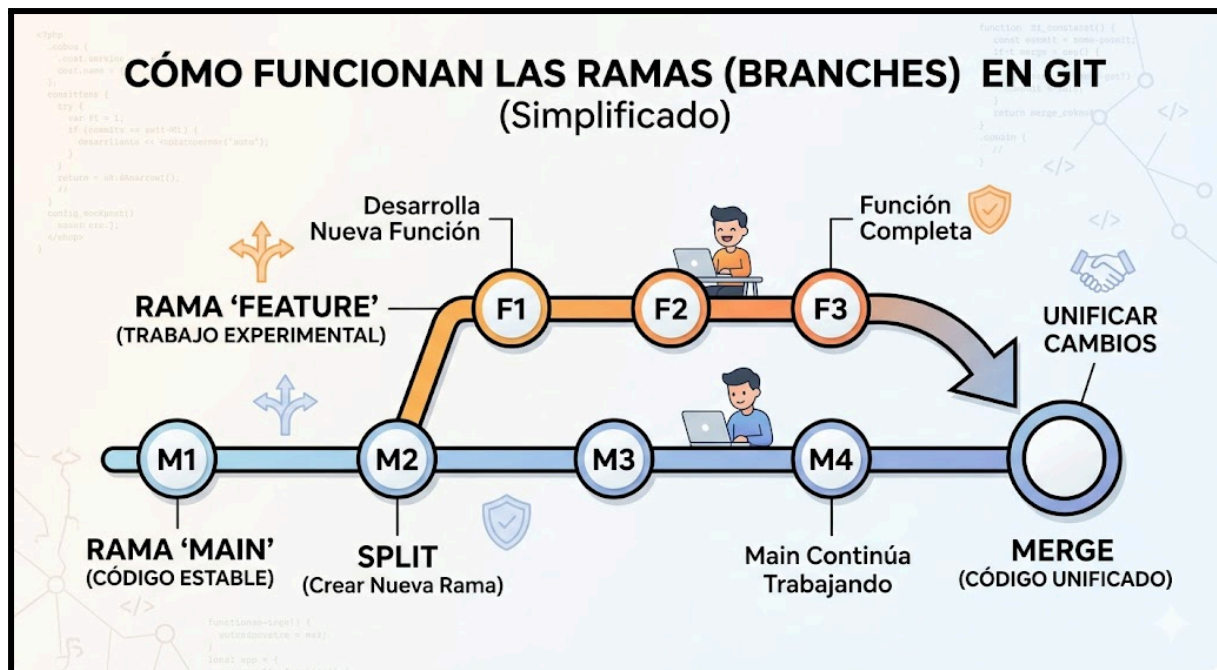


Figura 4. Ramas o Branch: Experimenta seguro sin romper el código principal .

2.6.1. El concepto de las ramas.

En Git, una rama es una línea de desarrollo independiente. Es como crear un "universo paralelo" o una línea temporal alternativa de tu código.

- **La Rama main:** Como vimos antes, esta es la línea temporal principal. Aquí es donde vive el código oficial, el que funciona perfectamente.
- **Crear rama:** `git branch <nombre>` crea una nueva rama apuntando al commit actual.
- **Cambiar de rama:** `git switch <nombre>` (o `git checkout <nombre>`) te mueve a esa rama para empezar a trabajar en ella.
- **Trabajar de forma aislada:** A partir de ese momento, todos los *commits* (guardados) que hagas se irán a esa nueva rama. El código en main se queda congelado y a salvo.
- **La fusión (Merge):** Cuando terminas de trabajar en tu rama, lo has probado todo y estás seguro de que funciona genial, le dices a Git: *"Coge todos los cambios de mi rama paralela y fusionarlos (merge) con la rama main"*.

2.6.2. ¿Por qué son tan importantes?

Experimentación sin miedo, puedes probar ideas locas. Si rompes el código, no pasa nada, solo has roto esa rama específica, no el proyecto principal. También, el **trabajo en equipo**, es la verdadera magia. Si sois 10 programadores, cada uno puede crear su propia rama para trabajar en una tarea distinta al mismo tiempo. Nadie se estorba, nadie

sobrescribe el trabajo del otro. Al final del día, el líder del proyecto va revisando y "fusionando" las ramas de todos en la rama main principal.

2.7. Fusión (Merge) y conflictos.

Una vez terminas de trabajar en una rama, debes unirla a la principal.

- **Merge:** Estando en la rama destino (ej. main), ejecutas `git merge <rama-a-fusionar>`. Esto trae los cambios de la otra rama a la actual.
- **Conflictos:** Si dos personas (o tú mismo en dos ramas) modifican **la misma línea del mismo archivo**, Git no sabrá con cuál quedarse. Se detendrá y te pedirá que elijas manualmente qué código conservar. Una vez arreglado el archivo, debes hacer un `git add` y un nuevo `git commit` para finalizar la fusión.

2.8. Herramientas avanzadas y seguridad.

El curso destaca comandos para situaciones complejas:

- **Stash (Guardado temporal):** Si estás trabajando en algo "sucio" (a medias) y necesitas cambiar de rama urgentemente, no puedes hacer commit de código roto. `git stash` guarda tus cambios en un "limbo" temporal y deja tu directorio limpio. Luego puedes recuperarlos con `git stash pop`.
- **Reset:**
 - `git reset --soft`: deshace el commit pero mantiene tus cambios en el área de preparación.
 - `git reset --hard <Hash>`: **Peligroso**. Borra todo lo que hiciste después de ese punto y regresa el proyecto exactamente a ese momento. Se pierden los cambios no guardados.
- **Reflog (El salvavidas):** Si borraste un commit o hiciste un reset hard por error, `git reflog` muestra un historial de *todos* tus movimientos (incluso los deshechos). Permite recuperar cosas que parecían perdidas buscando su Hash ahí.
- **Tags (Etiquetas):** Se usan para marcar puntos importantes, como versiones de lanzamiento (v1.0). `git tag -a v1.0 -m "Versión 1"`.

2.9. Resumen de comandos clave (Cheat Sheet).

A continuación resumimos los principales comandos con los que vamos a tratar más comúnmente:

- `git init`: Iniciar repo.
- `git add .`: Preparar cambios.
- `git commit -m "msg"`: Guardar cambios.
- `git status`: Ver estado.

- `git log`: Ver historial.
- `git switch <rama>`: Cambiar rama.
- `git merge <rama>`: Fusionar rama.
- `git restore <archivo>` (o `checkout`): Descartar cambios locales.

3. GITHUB.

3.1. Los orígenes: El problema de compartir código.

Como vimos antes, Linus Torvalds creó Git en 2005 y fue una revolución técnica, pero tenía un problema: colaborar por internet usando Git seguía siendo un proceso tosco y poco amigable. Normalmente, los desarrolladores tenían que enviarse parches de código por correo electrónico.

Ahí es donde entraron **Chris Wanstrath**, **PJ Hyett** y **Tom Preston-Werner**. En **2008**, lanzaron GitHub con una premisa brillante: crear una interfaz web bonita y fácil de usar sobre la tecnología de Git. Su lema original lo decía todo: *"Social Coding"* (Programación social).



Figura 5. P.J. Hyett (izquierda), Tom Preston-Werner (centro) y Chris Wanstrath.

3.2. La revolución de GitHub.

GitHub cambió para siempre la forma en la que se construye el software por dos grandes innovaciones:

- **El "Fork" (Bifurcación) de un clic:** Antes, contribuir a un proyecto de código abierto era un proceso burocrático. GitHub permitió que cualquier persona, con un solo clic, hiciera una copia exacta (un *fork*) del proyecto de otra persona, trabajara en él y propusiera mejoras.
- **Los "Pull Requests":** Fue su gran invención. Una vez que mejorabas el código en tu copia, enviabas un *Pull Request* al creador original. Era una forma educada y visual de decir: *"He arreglado este error, aquí tienes los cambios por si quieres integrarlos en tu proyecto principal"*. Esto permitía discutir el código línea por línea antes de aceptarlo.

GitHub se convirtió rápidamente en una especie de "red social" para programadores. Tu perfil de GitHub pasó a ser tu currículum real.

3.3. La era de Microsoft.

En **2018**, ocurrió un terremoto en la industria: **Microsoft compró GitHub por 7.500 millones de dólares**.

Al principio, hubo cierto pánico en la comunidad de código abierto (históricamente enfrentada a las políticas antiguas de Microsoft), e incluso algunos proyectos migraron a GitLab. Sin embargo, bajo el mando de Satya Nadella, Microsoft mantuvo la independencia de GitHub y lo mejoró drásticamente, haciendo gratuitos los repositorios privados que antes eran de pago.

3.4. El presente y futuro.

Hoy en día, GitHub no es solo un "almacén" de código; es el corazón de la industria tecnológica:

- Aloja más de **100 millones de desarrolladores** y la inmensa mayoría del software de código abierto del mundo.
- Ha integrado **GitHub Actions**, que permite automatizar pruebas y despliegues directamente en la plataforma.
- Ha liderado la revolución de la IA con **GitHub Copilot**, un asistente virtual que ayuda a escribir código en tiempo real.

3.5. Conceptos fundamentales: Git vs. GitHub

Es crucial distinguir ambas herramientas antes de empezar:

- **Git:** Es el sistema de control de versiones que vive en tu ordenador (local). Se encarga del historial, las ramas y los commits.
- **GitHub:** Es una plataforma en la nube (remota) que aloja repositorios Git. Funciona como un punto de reunión para el código, permitiendo copias de seguridad y, sobre todo, la colaboración entre desarrolladores que están en distintos lugares.

3.6. Conectar con el repositorio remoto (GitHub).

Para respaldar tu código y colaborar, debes subirlo a la nube.

- **Crear repositorio en GitHub:** Ve a GitHub, crea un nuevo repositorio (sin añadir README ni .gitignore por ahora para evitar conflictos iniciales).

Vincular repositorio local y remoto: Copia la URL SSH que te da GitHub y ejecuta en tu terminal:

```
git remote add origin git@github.com:Usuario/mi-primer-proyecto.git
```

- "origin" es el nombre estándar para el servidor remoto.

Subir cambios (Push): Envía tus commits locales al servidor:

```
git push -u origin main
```

- La opción "-u" vincula tu rama local con la remota, facilitando futuros envíos con solo `git push`.

3.7. Repositorios Remotos

Un repositorio en GitHub es la versión en la nube de tu proyecto.

- **Crear Repo:** Desde la web de GitHub, creas un nuevo proyecto. Puedes iniciarlo vacío o con un archivo README.md.
- **Vinculación (Remote):** Para conectar tu carpeta local con la de la nube, usas el comando:
 - `git remote add origin <URL>`: Esto le dice a tu Git local que "origin" es el apodo para la dirección URL de tu repositorio en GitHub.
- **Git Push (Subir):** Envía tus commits locales al servidor. La primera vez se usa `git push -u origin main` para vincular la rama local `main` con la remota.

3.8. Ignorar Archivos Innecesarios (.gitignore).

Es vital no subir archivos temporales o sensibles. Crea un archivo llamado ".gitignore" en la raíz de tu proyecto y escribe dentro los nombres de carpetas o extensiones que Git debe ignorar (ej. `*.log`, `temp/`). Git dejará de rastrearlos automáticamente.

3.9. Autenticación con GitHub (Claves SSH).

Para conectar tu estación de trabajo local con GitHub de forma segura sin tener que introducir tu contraseña cada vez, lo ideal es usar claves SSH.

3.9.1. Las Claves SSH.

SSH significa *Secure Shell*, pero para entenderlo de forma sencilla, piénsalo como un sistema mágico de **Candado y Llave**.

En lugar de usar una contraseña, le pides a tu ordenador que fabrique un par de herramientas que están conectadas matemáticamente entre sí. Se crean juntas y son inseparables:

- **La Clave Pública (El candado):** Esta es la que subes y le entregas a GitHub. Es totalmente pública. No importa quién la vea o si se pierde, porque un candado por sí solo no sirve para entrar a ningún lado; solo sirve para bloquear algo.
- **La Clave Privada (La llave):** Esta se queda guardada en lo más profundo de tu ordenador personal. **Nunca** se la das a nadie, ni siquiera a GitHub. Es la única llave en todo el universo capaz de abrir ese candado específico.

3.9.2. ¿Cómo funciona en la práctica?

Cuando tu ordenador intenta enviar o descargar código, ocurre una conversación invisible e instantánea:

- **GitHub dice:** "Alguien quiere entrar. Voy a cerrar esta puerta con el **Candado** (Clave Pública) que me diste el otro día. Si de verdad eres tú, ábrelo."
- **Tu ordenador:** Usa su **Llave** (Clave Privada) para abrir la puerta en milisegundos.
- **GitHub confirma:** "Perfecto, eres tú. Adelante."

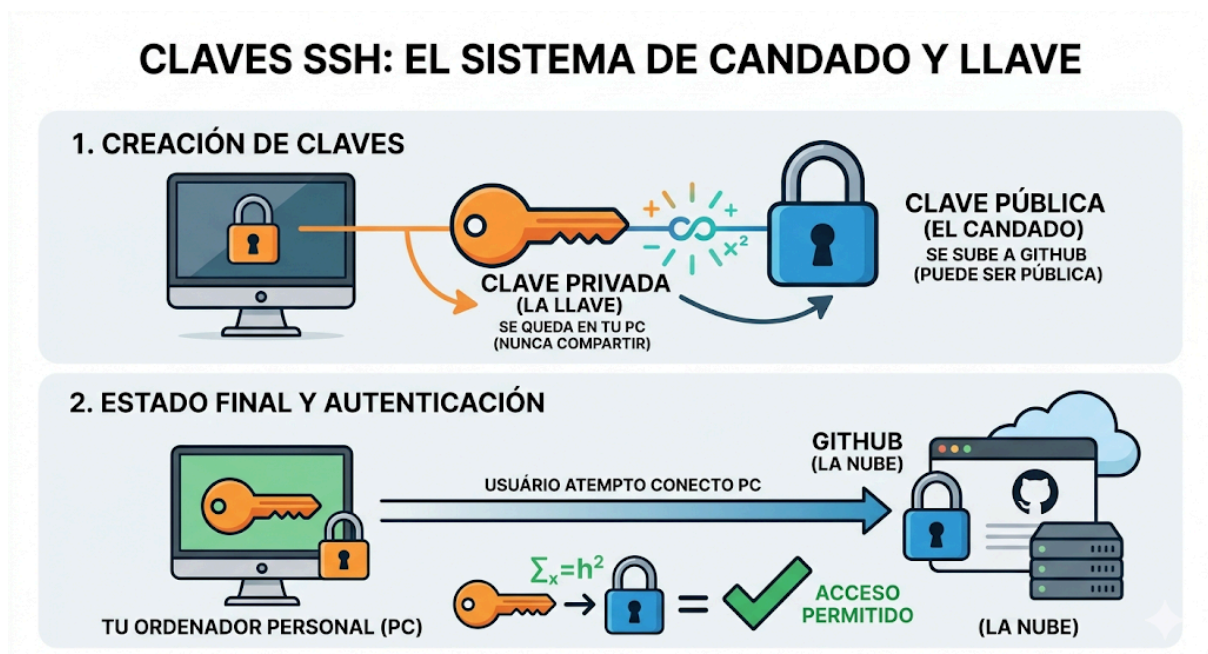


Figura 6. Candado y Llave: Cómo las claves SSH permiten el acceso seguro.

Estás dentro. No tuviste que teclear ninguna contraseña y ninguna información secreta viajó por internet donde un hacker pudiera espiarla. ¿Por qué a los programadores les encanta esto?

- **Comodidad:** Una vez configurado, tu ordenador hace todo el trabajo pesado de identificarse por ti.
- **Seguridad:** Si alguien quiere robar tu acceso, ya no le sirve adivinar una palabra; tendría que robar físicamente tu ordenador para conseguir la "Llave".

3.9.3. Cómo procedemos para crear y utilizar SSH.

Generar claves: En la terminal, ejecuta:

```
ssh-keygen -t ed25519 -C "tu@email.com"
```

- (Si tu sistema no soporta ed25519, usa `ssh-keygen -t rsa -b 4096`). Presiona enter para aceptar la ubicación por defecto y puedes dejar la "passphrase" vacía si lo deseas.
- **Añadir la clave a GitHub:** Copia el contenido de tu clave pública (el archivo que termina en `.pub`, usualmente en la carpeta oculta `.ssh`). Ve a GitHub -> Settings -> SSH and GPG keys -> New SSH key y pega el contenido allí.
- **Probar conexión:** Ejecuta `ssh -T git@github.com` para confirmar que GitHub te reconoce.

3.10. Sincronización de cambios (trabajo en equipo)

Cuando trabajas con otros o desde varios ordenadores, la versión de GitHub puede avanzar mientras tú trabajas en local.

- **Git Fetch:** Descarga el historial de cambios del servidor remoto a tu ordenador, pero **no** modifica tu código actual. Es una forma segura de "mirar" qué ha pasado sin tocar tu trabajo.
- **Git Pull:** Es la combinación de dos comandos: hace un *fetch* (descarga) y automáticamente un *merge* (fusión) de los cambios remotos en tu rama actual. Es el comando estándar para "actualizar" tu proyecto.
- **Conflictos en Remoto:** Si intentas hacer un *push* pero alguien más ha subido cambios antes que tú, GitHub rechazará tu subida. Primero debes hacer un *pull* para bajar sus cambios, fusionarlos con los tuyos (resolviendo conflictos si los hay) y luego hacer el *push*.

3.11. Clonación (Git Clone)

Si entras a un proyecto nuevo o cambias de ordenador, no necesitas iniciarlo de cero.

- **Comando:** `git clone <URL>` descarga una copia completa del repositorio (historial, ramas y archivos) desde GitHub a tu máquina. Esto configura automáticamente el remoto "origin".

3.12. Flujo de trabajo "Open source" (Fork y Pull request)

Este es el corazón de la colaboración en GitHub cuando no tienes permisos de escritura directa en un proyecto (como en proyectos de código abierto).

- **Fork (Bifurcación):** Es una copia de un repositorio ajeno (ej. el de un profesor o una librería famosa) que se guarda en **tu** cuenta de GitHub. Sobre este fork tú tienes control total para hacer cambios sin afectar al proyecto original.
- **Pull Request (PR):** Una vez has hecho cambios en tu fork, quieres que el dueño del proyecto original los acepte. Un *Pull Request* es una petición formal donde le dices: "He hecho estos cambios en mi copia, ¿por favor podrías 'tirar' (pull) de ellos para integrarlos en tu proyecto?".
 - Permite discusión, revisión de código y aprobación antes de fusionar.

3.13. Mantener un Fork actualizado

Si haces un fork, tu copia se queda congelada en el tiempo, pero el proyecto original sigue avanzando.

- **Sincronización:** Debes actualizar tu fork con los cambios del original para evitar conflictos futuros. GitHub ofrece un botón de "Sync Fork" o se puede hacer por consola añadiendo el repositorio original como un segundo remoto (usualmente llamado "upstream").

3.14. Documentación (Markdown)

GitHub renderiza automáticamente los archivos de texto para que se vean bonitos.

- **README.md:** Es la portada de tu proyecto. Se escribe en lenguaje **Markdown**, que usa símbolos simples para dar formato (ej. **#** para títulos, ****** para negritas). Es esencial para explicar qué hace tu proyecto y cómo instalarlo.

4. Otras herramientas de Github

Hasta ahora hemos dominado los fundamentos mecánicos de Git: cómo registrar cambios, viajar en el tiempo a través de nuestro historial y colaborar de forma segura mediante ramas. Sin embargo, GitHub es mucho más que un simple almacén de código en la nube; es una plataforma integral diseñada para potenciar tu identidad profesional y automatizar tu ciclo de desarrollo.

En este nuevo y breve capítulo, daremos un salto cualitativo y explicaremos superficialmente las herramientas avanzadas que transformarán un simple repositorio en un proyecto dinámico y completo.

4.1. Personalización del perfil (Profile README).

Históricamente, el perfil de GitHub mostraba de forma automática tus repositorios anclados y el clásico gráfico de tus contribuciones (los famosos "cuadrados verdes"). Sin embargo, la plataforma introdujo una característica especial: si creas un repositorio público cuyo nombre sea **exactamente igual a tu nombre de usuario** y le añades un archivo README.md, el contenido de ese archivo se renderiza automáticamente en la parte superior de tu página principal.

Es, a todos los efectos, una página de inicio personal, un portafolio y un currículum dinámico integrado directamente en el centro neurálgico del desarrollo de software.

4.1.1. Posibilidades y elementos clave.

Al basarse en Markdown (y admitir etiquetas HTML básicas), este archivo te permite escapar de la rigidez de un perfil estándar. Las posibilidades para destacar son inmensas:

- **Presentación Estructurada:** Puedes utilizar encabezados y bloques de texto para presentarte, detallar en qué proyectos estás trabajando actualmente o mencionar que estás compaginando tu jornada laboral con la finalización de un máster.
- **Stack Tecnológico Visual:** Mediante el uso de escudos (*badges*) e iconos SVG, puedes diseñar secciones muy limpias que muestran visualmente las herramientas y lenguajes que dominas, destacando tus habilidades clave, como la automatización y creación de scripts en Bash.
- **Estadísticas en Tiempo Real:** Existen proyectos de código abierto (como *GitHub Readme Stats*) que generan tarjetas dinámicas. Al insertar un simple enlace en tu Markdown, tu perfil mostrará automáticamente métricas actualizadas sobre tu rendimiento, los lenguajes que más utilizas o tu nivel de actividad.

4.1.2. Dominio de Markdown (.md).

Markdown es un **lenguaje de marcado ligero**. En términos muy sencillos, es una forma de dar formato a un texto (poner negritas, crear listas, añadir enlaces o imágenes) utilizando caracteres normales de tu teclado, sin necesidad de usar botones o menús complejos como harías en Microsoft Word o Google Docs.

Su filosofía principal es la **legibilidad**: un documento escrito en Markdown debe ser perfectamente comprensible para un humano al leerlo en texto plano, sin que las etiquetas de formato estorben a la vista.

4.1.2.1. Breve historia.

Fue creado en **2004** por **John Gruber** (con importantes aportes de Aaron Swartz). La idea de Gruber era construir una sintaxis que fuera fácil de escribir y leer en su forma original, pero que al mismo tiempo pudiera convertirse de forma automática e impecable a HTML (el lenguaje con el que se construyen las páginas web).



Figura 7. John Gruber estableció los estándares de la publicación independiente.

4.1.2.2. ¿Por qué es el estándar en el desarrollo de software?

Hoy en día, Markdown está en todas partes (foros, blogs, aplicaciones de notas) y es el rey indiscutible en el mundo de la programación por varias razones:

- **Es puro texto:** Como no tiene formatos ocultos ni metadatos extraños, pesa muy poco y se lleva perfectamente con sistemas de control de versiones como Git (es muy fácil ver qué línea cambió exactamente).

- **Es universal:** Un archivo `.md` se puede abrir y leer en cualquier sistema operativo y con cualquier editor de texto básico (desde el Bloc de notas hasta Visual Studio Code).
- **Conversión rápida:** Existen herramientas que transforman instantáneamente un archivo Markdown en un PDF, un libro electrónico (EPUB), una página web o una presentación de diapositivas.

4.1.2.3. Sintaxis básica.

Para que veas lo intuitivo que es, aquí tienes una pequeña tabla con los comandos más utilizados:

Elemento	Cómo se escribe en Markdown	Qué se genera
Títulos	<code># Título Principal</code> <code>## Subtítulo</code>	Encabezados de diferentes tamaños (como H1, H2 en HTML).
Negrita	<code>**Texto importante**</code>	Texto importante
Cursiva	<code>*Texto en cursiva*</code>	<i>Texto en cursiva</i>
Listas	<code>- Elemento 1</code> <code>- Elemento 2</code>	Una lista con viñetas estándar.
Enlaces	<code>[Texto visible](https://url.com)</code>	Un enlace en el que se puede hacer clic.
Código	<code>`código en línea`</code> o bloques con <code>```</code>	Resalta el texto con tipografía de programación.

4.1.2.4. Markdown en GitHub (GFM).

Como comentábamos, Markdown es un estándar excelente, pero GitHub se dio cuenta de que los programadores necesitaban herramientas específicas que el diseño original de John Gruber no contemplaba. Por ello, crearon su propia variante ampliada: **GitHub Flavored Markdown (GFM)**.

Mientras que el Markdown clásico estaba pensado principalmente para publicar artículos legibles en la web, GFM está diseñado específicamente para facilitar el trabajo técnico, la documentación de proyectos y la colaboración en equipo.

4.1.2.4.1. Los "superpoderes" que añade GFM.

GFM toma la base sencilla de Markdown y le añade características que hoy en día consideramos imprescindibles en cualquier repositorio:

- **Resaltado de sintaxis en bloques de código:** En el Markdown original, el código solía verse como texto plano. GFM permite indicar qué lenguaje estás utilizando para que aplique los colores correspondientes de forma automática. Al redactar manuales o documentación técnica, esto marca una diferencia visual enorme.

Bash

```
# GFM detecta que esto es Bash y colorea la sintaxis
echo "Automatizando tareas..."
sudo apt update
```

4.2. GitHub Pages.

GitHub Pages es un servicio de alojamiento web gratuito y directo que ofrece la propia plataforma. En lugar de tener que alquilar un servidor externo y lidiar con configuraciones complejas, GitHub toma los archivos de tu repositorio y los publica automáticamente como una página web real y accesible para todo el mundo.

Básicamente, transforma un almacén de código estático en un sitio web en vivo, que normalmente queda disponible bajo una dirección web del tipo "www.tu-usuario.github.io".

4.2.1. Casos de uso ideales.

Este servicio está diseñado específicamente para alojar "sitios web estáticos" (páginas que no requieren procesar bases de datos en tiempo real, como un foro o una tienda online). Es una herramienta extremadamente potente para escenarios específicos:

- **Portafolios profesionales:** Es el lugar ideal para desplegar una web personal que sirva de escaparate de tus habilidades. Mostrar un portafolio en vivo, interactivo y limpio es un recurso diferencial cuando te encuentras en una búsqueda activa de

nuevas oportunidades laborales, permitiendo a los reclutadores (ya sean de empresas locales en Barcelona o para puestos en remoto) ver tus capacidades en acción al instante.

- **Documentación técnica:** Si estás estructurando manuales de desarrollo, redactando apuntes de introducción a la Inteligencia Artificial o creando guías de comandos en Bash, GitHub Pages te permite publicar esos documentos como un sitio web formal, organizado y muy fácil de navegar para la comunidad.
- **Páginas de presentación de proyectos:** Permite crear una *landing page* (página de aterrizaje) atractiva para explicar de forma visual cómo funciona una herramienta de código abierto que hayas creado.

4.2.2. La magia detrás: De Markdown a web.

Lo más interesante de GitHub Pages es que se integra a la perfección con todo lo que hemos visto hasta ahora. Cuenta con un motor interno (llamado **Jekyll**) que entiende el formato **Markdown**.

Esto significa que no es estrictamente necesario que programes toda la web desde cero en HTML y CSS. Puedes redactar toda la información de tu proyecto en simples archivos `.md`, y GitHub Pages se encargará de ensamblarlos, aplicarles un diseño visual limpio y publicarlos como una página web terminada. Si en el futuro modificas una línea en tu archivo Markdown y haces un *commit*, la web pública se actualizará sola en cuestión de segundos.

4.3. GitHub Actions (Automatización).

GitHub Actions es la plataforma de **automatización y CI/CD** (Integración y Despliegue Continuo) integrada directamente dentro de GitHub.

En términos sencillos, es como tener un equipo de asistentes o "robots" trabajando las 24 horas dentro de tu repositorio. Puedes programarlos para que ejecuten tareas automáticas cada vez que ocurra algo específico en tu proyecto (por ejemplo, cuando alguien sube código nuevo, cuando se aprueba un cambio, o incluso a una hora exacta todos los días).

4.3.1. ¿Cómo funciona la automatización? (Conceptos clave).

Todo el sistema se configura mediante archivos de texto muy estructurados escritos en un formato llamado **YAML** (.yml), los cuales se guardan en una carpeta especial (.github/workflows) dentro de tu repositorio.

Un flujo de trabajo se compone de estos elementos básicos:

- **Eventos (Triggers):** Es el detonante. Le dice al sistema *cuándo* debe empezar a trabajar. Puede ser un comando `git push` a la rama principal, o la creación de un nuevo *Pull Request*.
- **Flujos de trabajo (Workflows):** Es el proceso completo que has definido. Un repositorio puede tener varios flujos de trabajo diferentes.
- **Trabajos (Jobs):** Un flujo de trabajo está dividido en trabajos. Por defecto, si hay varios, se ejecutan a la vez (en paralelo). Por ejemplo, un trabajo puede compilar tu proyecto y otro distinto puede revisar la ortografía de tu documentación al mismo tiempo.
- **Pasos (Steps) y Acciones (Actions):** Cada trabajo es una lista de pasos. Aquí es donde ocurre la acción real. Puedes decirle que ejecute comandos directos en la consola, o usar "Acciones" ya creadas y compartidas por la comunidad de GitHub para ahorrar tiempo.

4.3.2. Casos de uso más potentes.

Esta herramienta ha cambiado la forma de desarrollar software por la cantidad de trabajo manual que elimina. Los usos más comunes son:

- **Pruebas automáticas (Testing):** Cada vez que alguien propone un cambio de código, GitHub Actions levanta un pequeño servidor virtual en la nube en segundos, descarga el código y ejecuta pruebas automáticas. Así, te aseguras de que el nuevo código no rompe nada antes de fusionarlo con la rama main.
- **Despliegue Continuo (Deploy):** Puedes configurarlo para que, en el momento en que confirmes un cambio importante, la acción empaquete tu proyecto y lo publique directamente en internet o en los servidores de tu empresa, sin que tú tengas que tocar nada más.
- **Gestión del repositorio:** También sirve para tareas administrativas, como etiquetar problemas (*issues*) de forma automática, enviar mensajes a servicios de chat externos o cerrar discusiones antiguas.

Existen multitud de "acciones" prefabricadas para integrar con servicios como Amazon AWS, Google Cloud o Azure.

Aunque es una característica muy potente, GitHub ofrece una cantidad generosa de minutos gratuitos (2000 minutos/mes) para repositorios públicos y privados, lo cual es suficiente para la mayoría de proyectos personales.

5. Bibliografía.

- Chacon, S., & Straub, B. (2014, 1 1). *Pro Git (2ª ed.)*. Pro Git. <https://git-scm.com/book/es/v2>
- GitHub. (2026, 6 2). *Conexión a GitHub con SSH - Documentación de GitHub*. GitHub Docs. Retrieved June 2, 2026, from <https://docs.github.com/es/authentication/connecting-to-github-with-ssh>
- GitHub. (2026, 6 2). *Documentación de GitHub Pages - Documentación de GitHub*. GitHub Docs. Retrieved June 2, 2026, from <https://docs.github.com/es/actions>
- GitHub. (2026, 6 2). *Documentación de GitHub Pages - Documentación de GitHub*. GitHub Docs. Retrieved June 2, 2026, from <https://docs.github.com/es/pages>
- GitHub. (2026, 6 2). *Sintaxis básica de redacción y formato - Documentación de GitHub*. GitHub Docs. Retrieved June 2, 2026, from <https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>
- Gruber, J. (2004, 1 1). *Markdown*. Daring Fireball. Retrieved June 2, 2026, from <https://daringfireball.net/projects/markdown/>
- Hazra, A. (2020, 1 1). *GitHub - anuraghazra/github-readme-stats: :zap: Dynamically generated stats for your github readmes*. GitHub. Retrieved June 2, 2026, from <https://github.com/anuraghazra/github-readme-stats>
- Microsoft. (2018, June 4). *Microsoft to acquire GitHub for \$7.5 billion - Source*. Microsoft Source. Retrieved June 2, 2026, from <https://news.microsoft.com/2018/06/04/microsoft-to-acquire-github-for-7-5-billion/>
- Moure, B. (2024). *Curso Git y Github*. mouredev. <https://moure.dev/>
- Preston-Werner, T. (2018, 04 10). *GitHub is Open for Business!* The GitHub Blog. <https://www.google.com/search?q=https://github.blog/2008-04-10-github-is-open-for-business/>